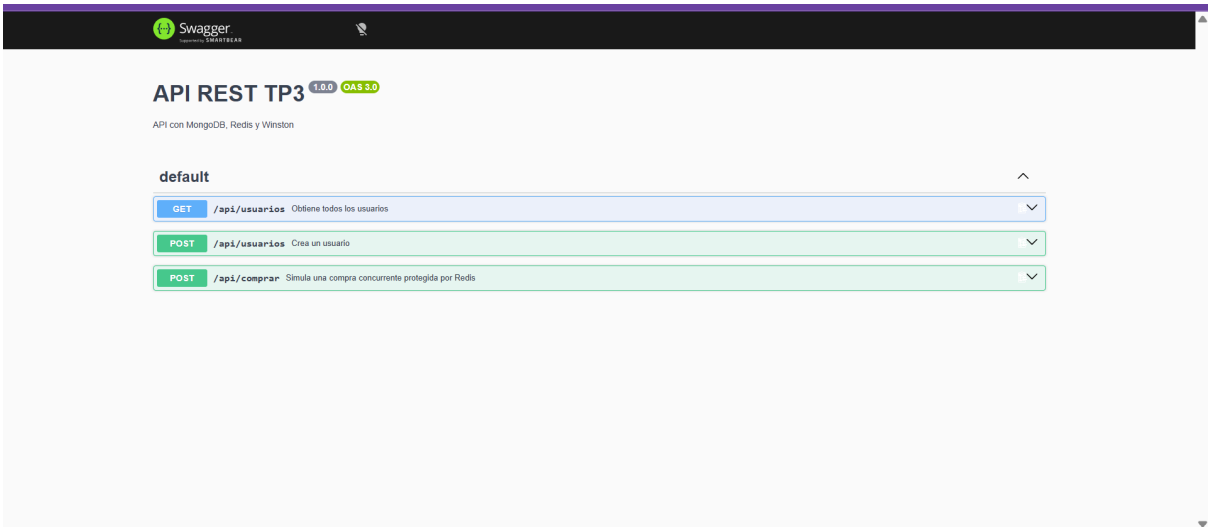
Parte 1:
punto 3:

```
PS C:\xampp\htdocs\ProyectosFacultad\BaseDatos2\programacionDistribuida\tp3> npm start

> tp3@1.0.0 start
> node index.js

{"level":"info","message":"Servidor HTTPS iniciado en https://localhost:3443","timestamp":"2026-09-03T22:57:30.451Z"}
{"level":"info","message":"Conectado a Redis/Memurai exitosamente","timestamp":"2026-09-03T22:57:30.502Z"}
{"level":"info","message":"Conectado a MongoDB exitosamente","timestamp":"2026-09-03T22:57:30.515Z"}
{"level":"info","message":"Inventario inicial creado en MongoDB","timestamp":"2026-09-03T22:57:30.708Z"}
```



default

GET /api/usuarios Obtiene todos los usuarios

Parameters Cancel

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'https://localhost:3443/api/usuarios' \
  -H 'accept: */*'
```

Request URL

```
https://localhost:3443/api/usuarios
```

Server response

Code	Details
200	Response body

```
[
  {
    "_id": "6a99fd89809656c2d4f6efd3",
    "nombre": "Ana",
    "apellido": "Gómez",
    "dni": "11223344",
    "telefono": "2611234567",
    "mail": "ana@email.com",
    "__v": 0
  }
]
```

Download

Response headers

```
connection: keep-alive
content-length: 143
content-type: application/json; charset=utf-8
date: Thu,03 Sep 2026 23:14:14 GMT
etag: W/"8f-6OXc+T2Tt20aTks1LZUXuMQOFVw"
keep-alive: timeout=5
x-powered-by: Express
```

Responses

Code	Description	Links
200	Éxito	*No links*

POST /api/comprar Simula una compra concurrente protegida por Redis

Parameters Cancel

No parameters

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'https://localhost:3443/api/comprar' \
  -H 'accept: */*' \
  -d ''
```

Request URL

```
https://localhost:3443/api/comprar
```

Server response

Code	Details
200	Response body

```
{
  "mensaje": "Compra exitosa",
  "stockRestante": 0
}
```

Download

Response headers

```
connection: keep-alive
content-length: 46
content-type: application/json; charset=utf-8
date: Thu,03 Sep 2026 23:15:47 GMT
etag: W/"2e-DeWiASHza7nI+ELNyaOzfiwdFRE"
keep-alive: timeout=5
x-powered-by: Express
```

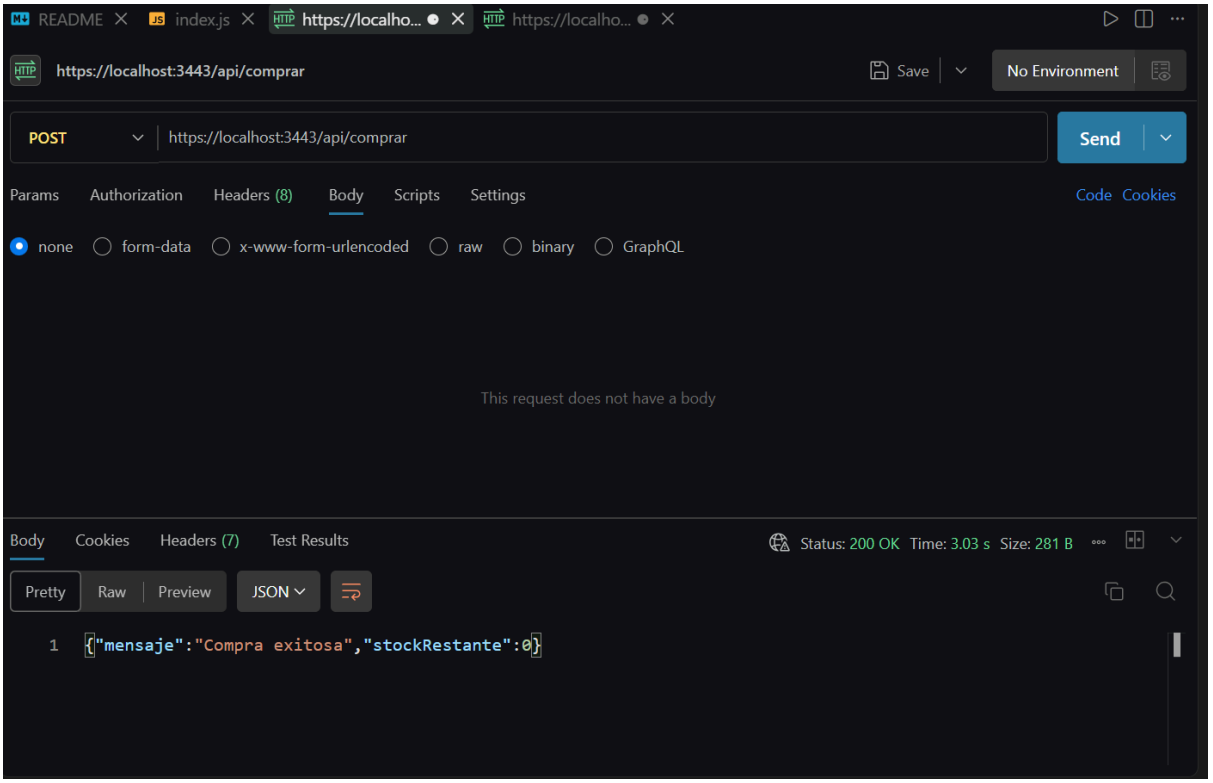
Responses

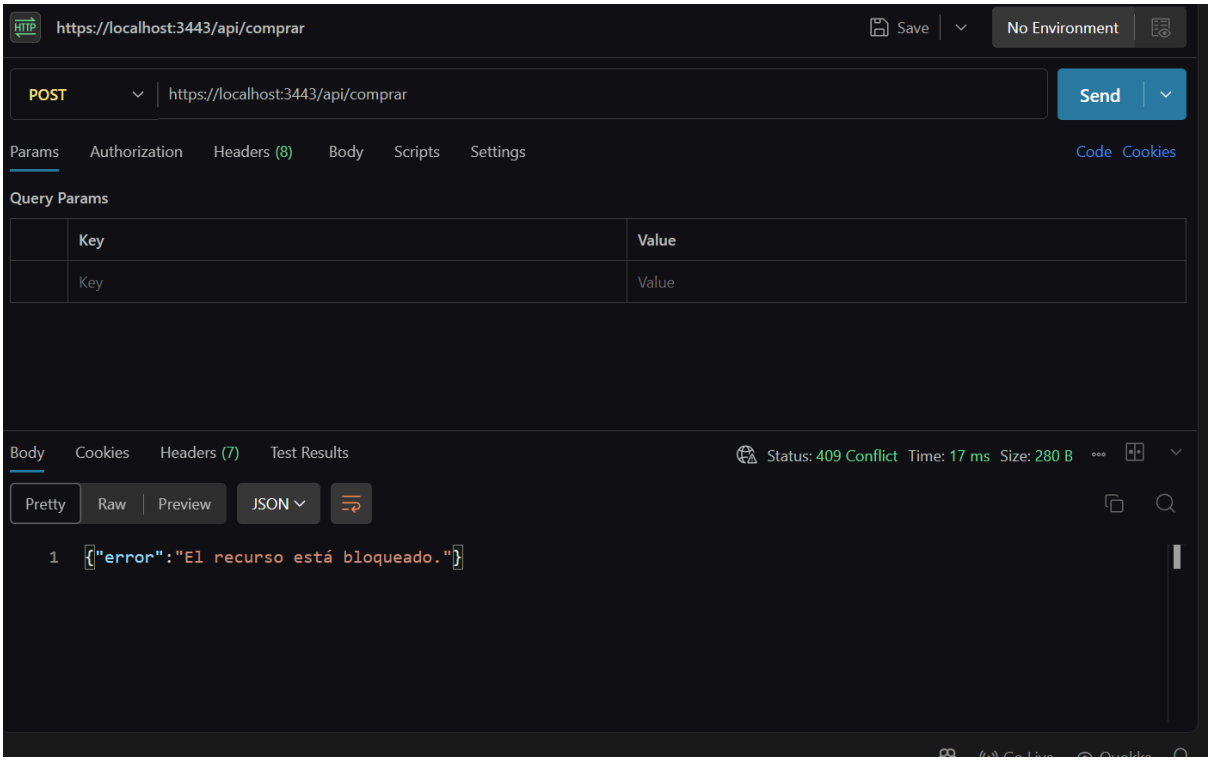
Code	Description	Links
200	Compra exitosa	*No links*
400	Stock insuficiente	*No links*
409	Recurso bloqueado	*No links*

Dar formato al texto ☑

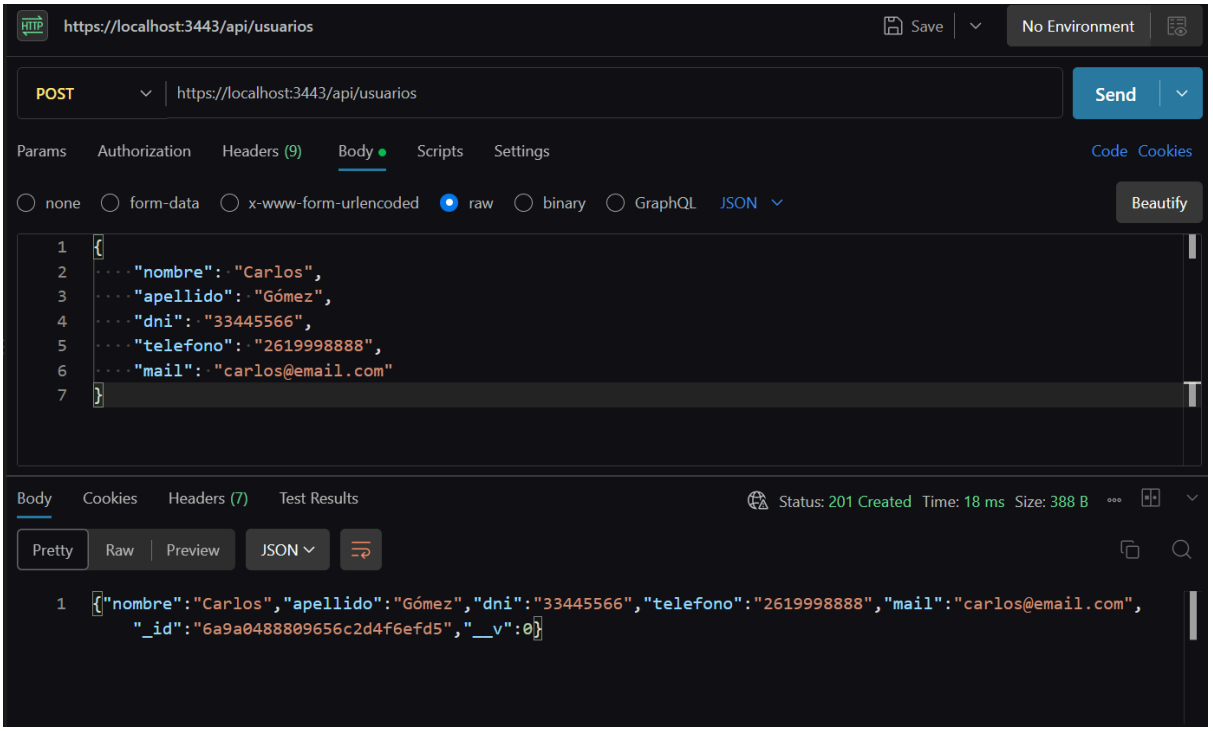

```
{
  "openapi": "3.0.0",
  "info": {
    "title": "API REST TP3",
    "version": "1.0.0",
    "description": "API con MongoDB, Redis y Winston"
  },
  "paths": {
    "/api/usuarios": {
      "get": {
        "summary": "Obtiene todos los usuarios",
        "responses": {
          "200": {
            "description": "Éxito"
          }
        }
      },
      "post": {
        "summary": "Crea un usuario",
        "responses": {
          "201": {
            "description": "Creado"
          },
          "400": {
            "description": "Bad Request"
          },
          "409": {
            "description": "Conflicto (DNI duplicado)"
          }
        }
      }
    },
    "/api/comprar": {
      "post": {
        "summary": "Simula una compra concurrente protegida por Redis",
        "responses": {
          "200": {
            "description": "Compra exitosa"
          },
          "400": {
            "description": "Stock insuficiente"
          },
          "409": {
            "description": "Recurso bloqueado"
          }
        }
      }
    }
  }
}
```

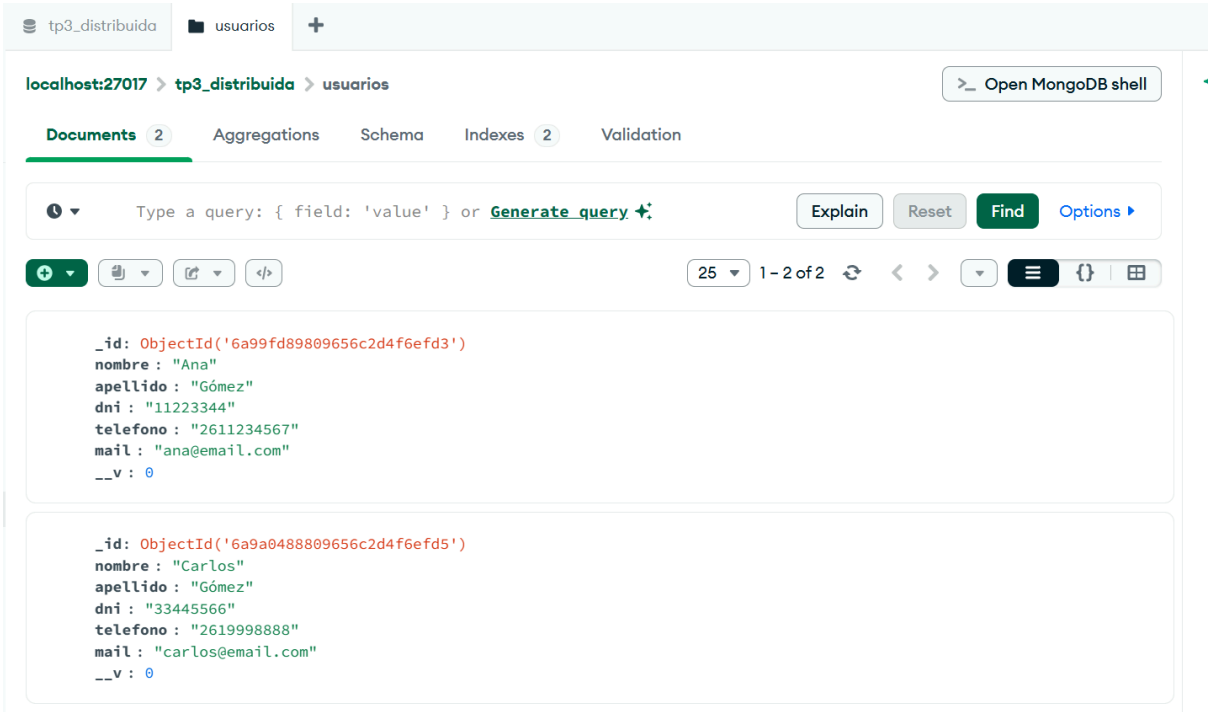

punto 4:


README
index.js
https://localho...
https://localho...
https://localhost:3443/api/comprar
Save
No Environment
POST
https://localhost:3443/api/comprar
Send
Params
Authorization
Headers (8)
Body
Scripts
Settings
Code
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
This request does not have a body
Body
Cookies
Headers (7)
Test Results
Status: 200 OK
Time: 3.03 s
Size: 281 B
Pretty
Raw
Preview
JSON
1 {"mensaje":"Compra exitosa","stockRestante":0}

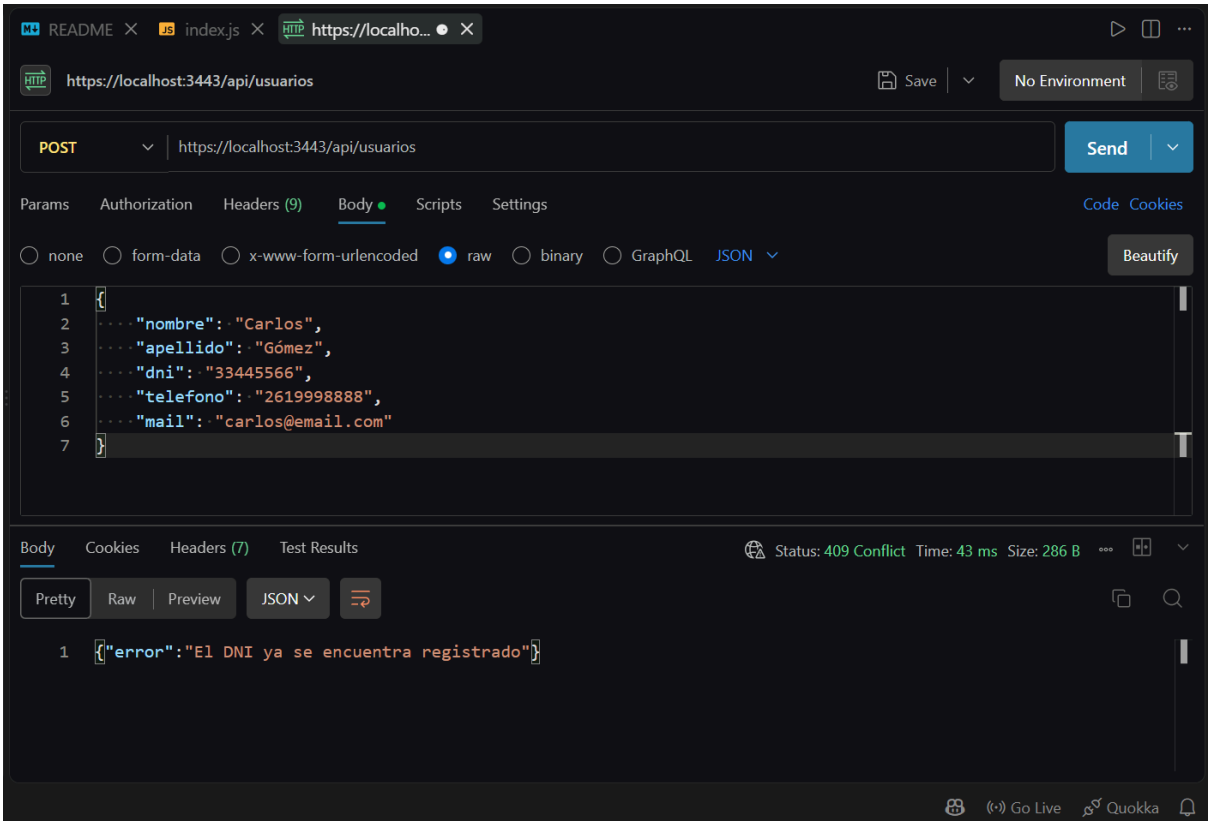

https://localhost:3443/api/comprar
Save
No Environment
POST
https://localhost:3443/api/comprar
Send
Params
Authorization
Headers (8)
Body
Scripts
Settings
Code
Cookies
Query Params
Key
Value
Key
Value
Body
Cookies
Headers (7)
Test Results
Status: 409 Conflict
Time: 17 ms
Size: 280 B
Pretty
Raw
Preview
JSON
1 {"error":"El recurso está bloqueado."}

punto 6:
Creación documento:

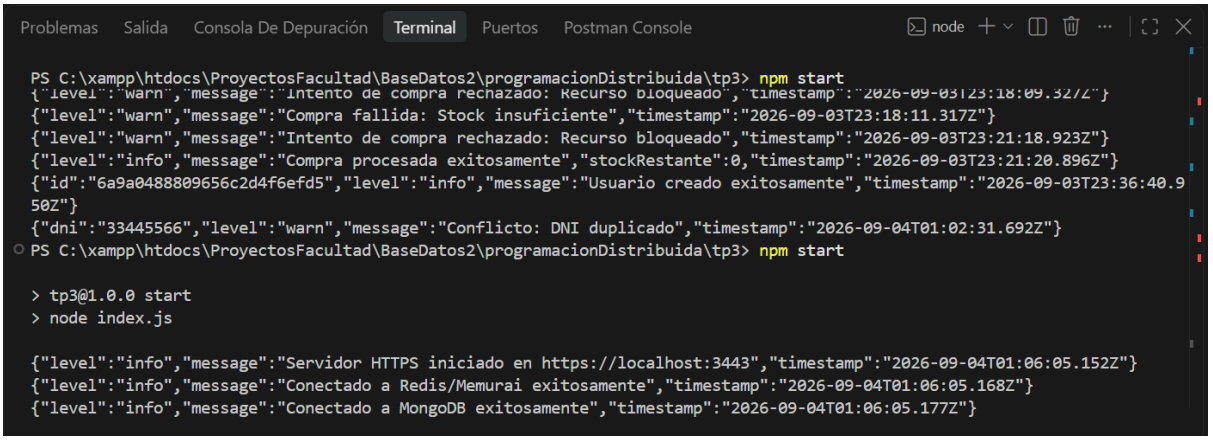


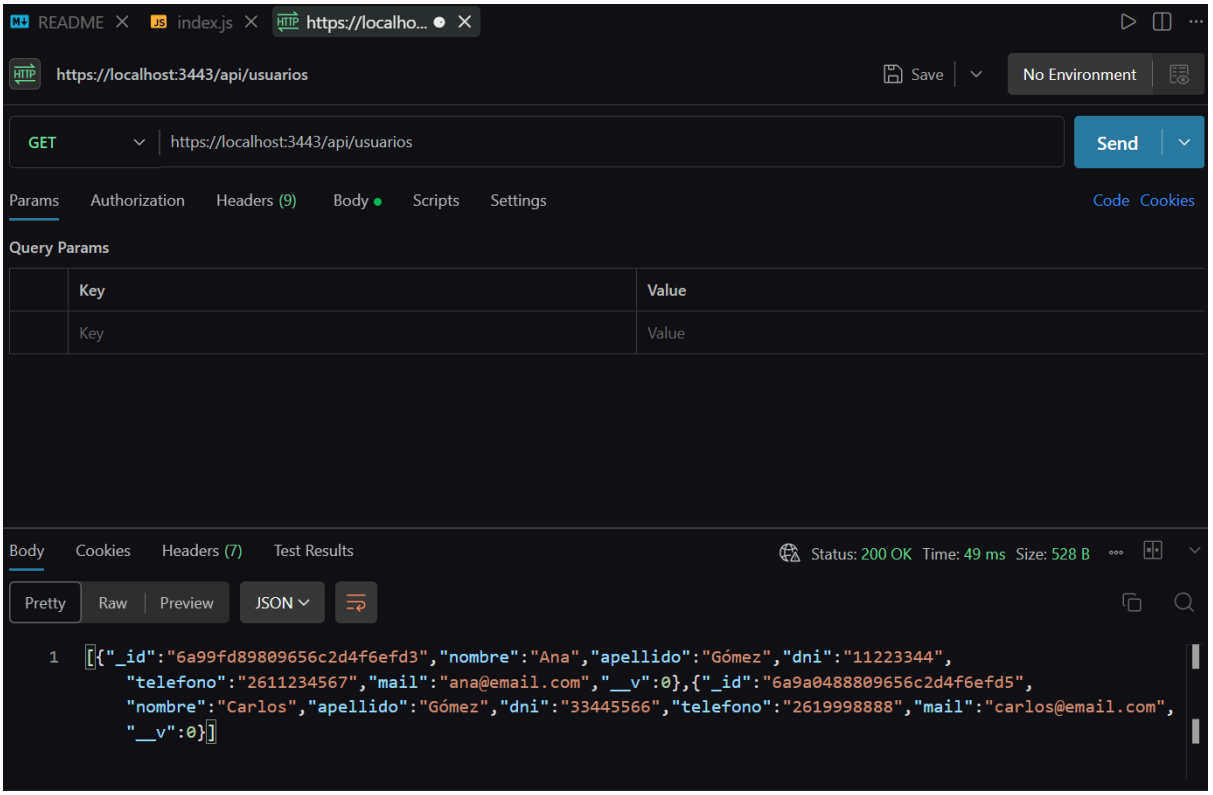


Forzado rechazo de datos:



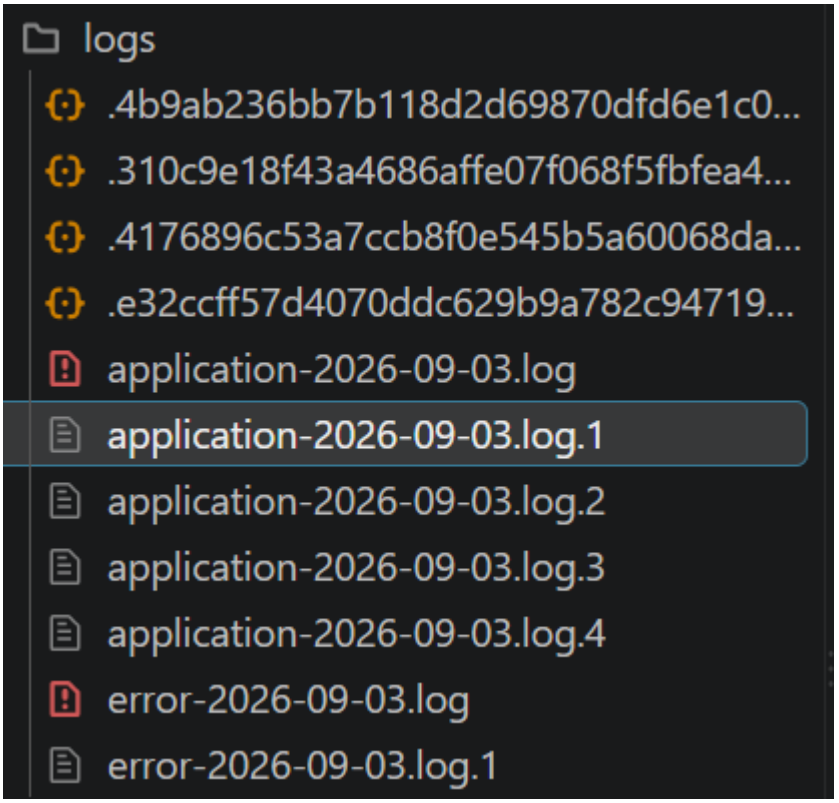
Comprobar persistencia:

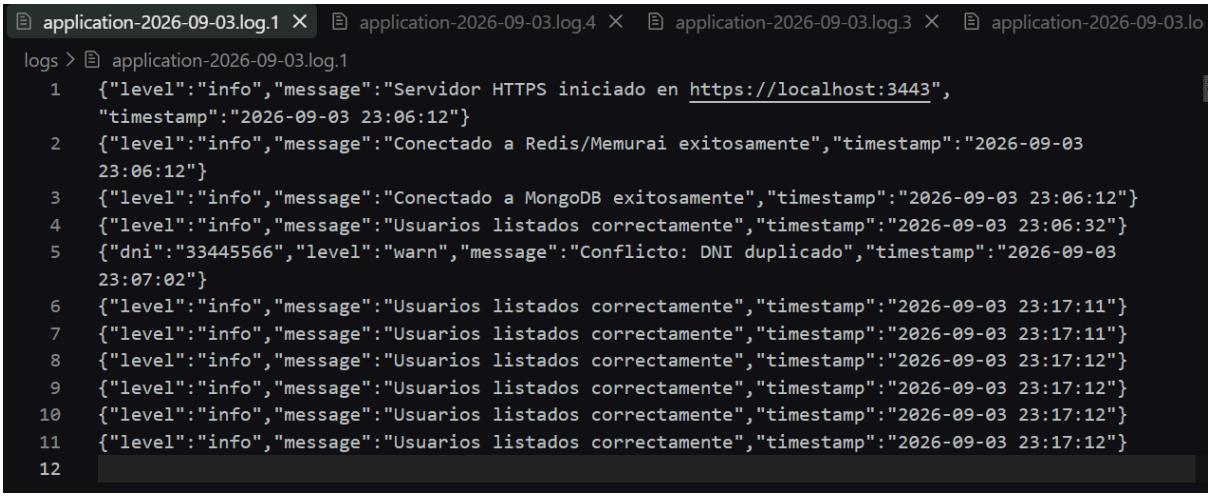




- ¿Qué diferencia existe entre almacenar un recurso en un arreglo JavaScript y almacenarlo en MongoDB?
- Almacenar en un arreglo es volátil, si la aplicación Node.js se reinicia, todos los datos se pierden. Almacenar en MongoDB otorga persistencia permanente a los datos en el disco físico, permitiendo que la información sobreviva a reinicios del servidor y escale adecuadamente.
- ¿Qué función cumplen el Schema y el Model de Mongoose dentro de la aplicación?
- El Schema define la estructura, los tipos de datos y las restricciones del documento. El Model es el envoltorio funcional que toma ese Schema y nos provee los métodos necesarios para interactuar de forma sencilla con la base de datos.
- ¿Por qué conviene diferenciar un identificador inválido de un identificador válido cuyo documento no existe?
- Para devolver el código de error semánticamente correcto según REST. Un identificador inválido es un error de formato en la petición del cliente y debe devolver 400 Bad Request. Un identificador válido pero que no existe en la base de datos significa que la petición está bien estructurada, pero el recurso específico no fue hallado, por lo que debe devolver 404 Not Found.

Parte 2:
punto 9:





README index.js .env error-2026-09-03.log https://localho... logger.js

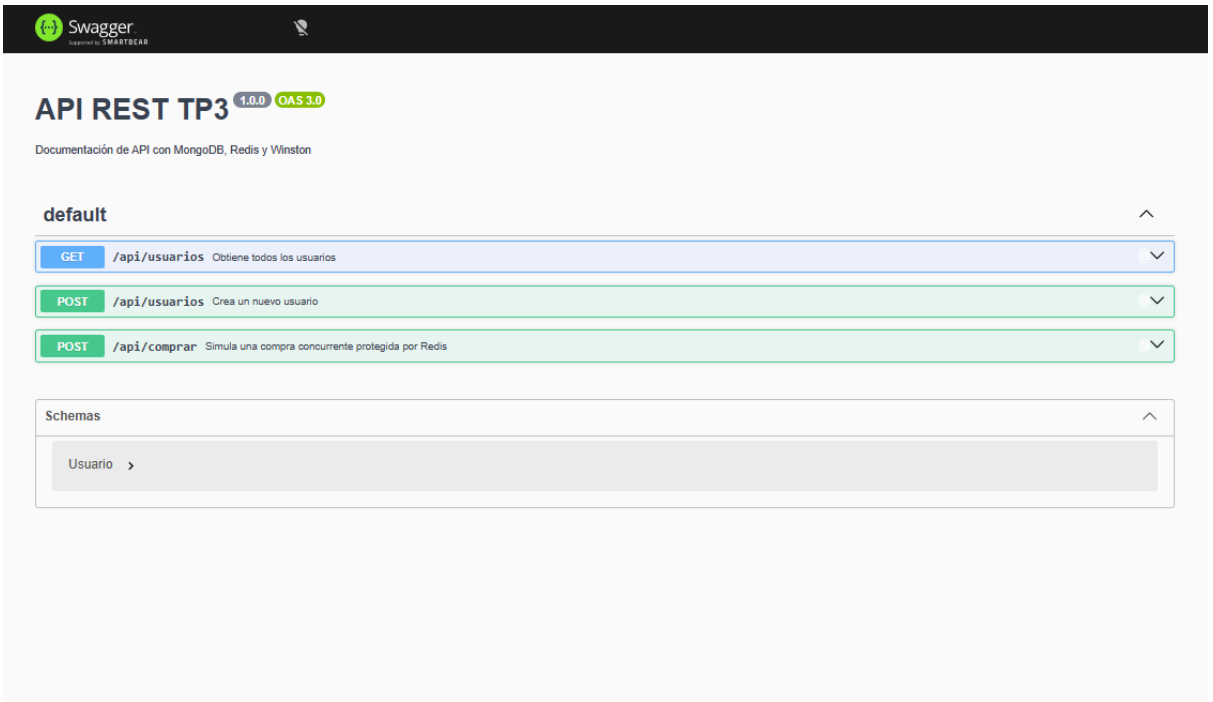
logs > error-2026-09-03.log

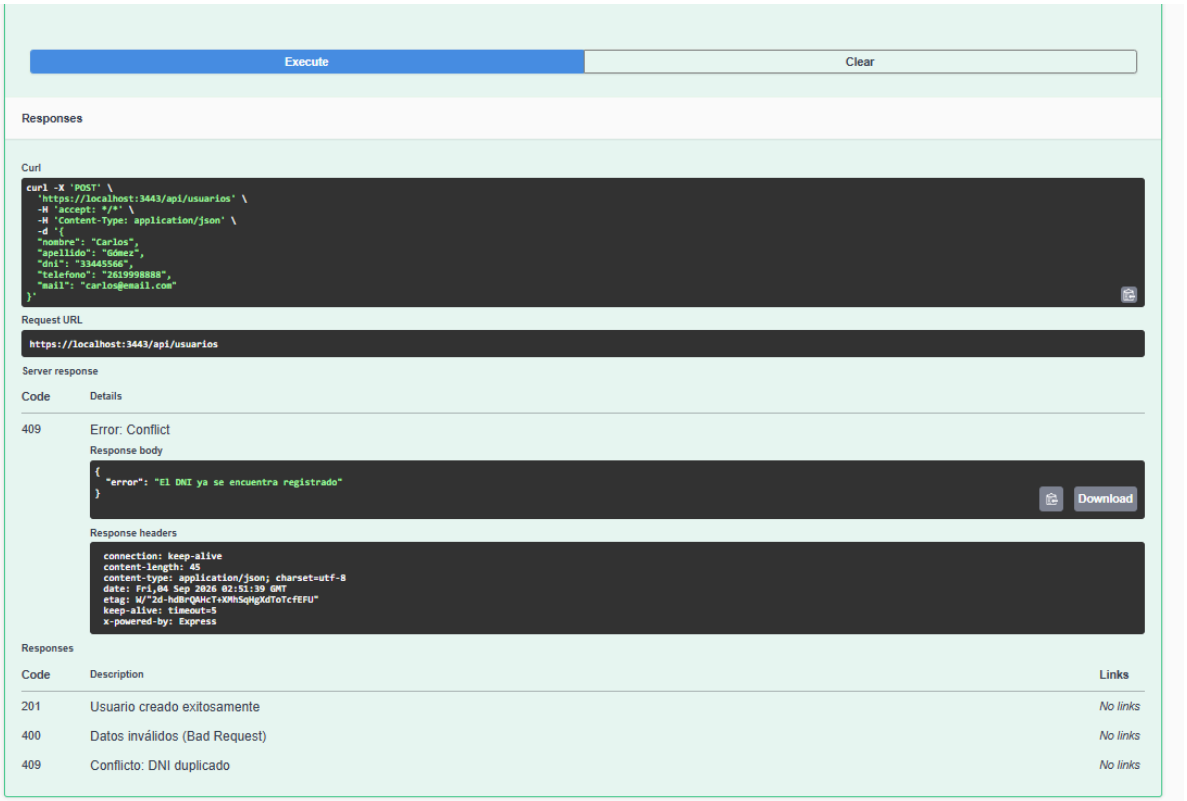

```
{"error":"connect ECONNREFUSED 127.0.0.1:9999","level":"error","message":"Error en Redis","timestamp":"2026-09-03 23:19:13"}
{"error":"connect ECONNREFUSED 127.0.0.1:9999","level":"error","message":"Error en Redis","timestamp":"2026-09-03 23:19:13"}
{"error":"connect ECONNREFUSED 127.0.0.1:9999","level":"error","message":"Error en Redis","timestamp":"2026-09-03 23:19:14"}
{"error":"connect ECONNREFUSED 127.0.0.1:9999","level":"error","message":"Error en Redis","timestamp":"2026-09-03 23:19:14"}
{"error":"connect ECONNREFUSED 127.0.0.1:9999","level":"error","message":"Error en Redis","timestamp":"2026-09-03 23:19:15"}
{"error":"connect ECONNREFUSED 127.0.0.1:9999","level":"error","message":"Error en Redis","timestamp":"2026-09-03 23:19:15"}
{"error":"connect ECONNREFUSED 127.0.0.1:9999","level":"error","message":"Error en Redis","timestamp":"2026-09-03 23:19:17"}
{"error":"connect ECONNREFUSED 127.0.0.1:9999","level":"error","message":"Error en Redis","timestamp":"2026-09-03 23:19:19"}
{"error":"connect ECONNREFUSED 127.0.0.1:9999","level":"error","message":"Error en Redis","timestamp":"2026-09-03 23:19:21"}
```

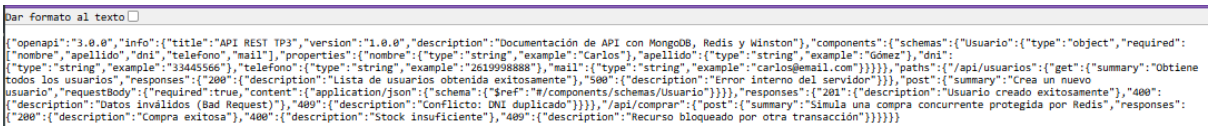

- ¿Qué diferencia práctica existe entre registrar un evento como INFO, WARN o ERROR?
- Permite filtrar y priorizar el diagnóstico. INFO se usa para seguir el flujo normal del sistema. WARN alerta sobre situaciones manejables o errores del cliente que no rompen el sistema. ERROR alerta sobre fallos críticos que requieren intervención técnica inmediata.
- ¿Qué problema resuelven la rotación y la retención limitada de archivos de log?
- Evitan que el servidor colapse por falta de espacio en disco. Sin rotación, un único archivo .log crecería hasta pesar gigabytes, volviéndose imposible de abrir o leer. La retención previene que el servidor se quede sin memoria de almacenamiento.

Parte 3:

punto 12:



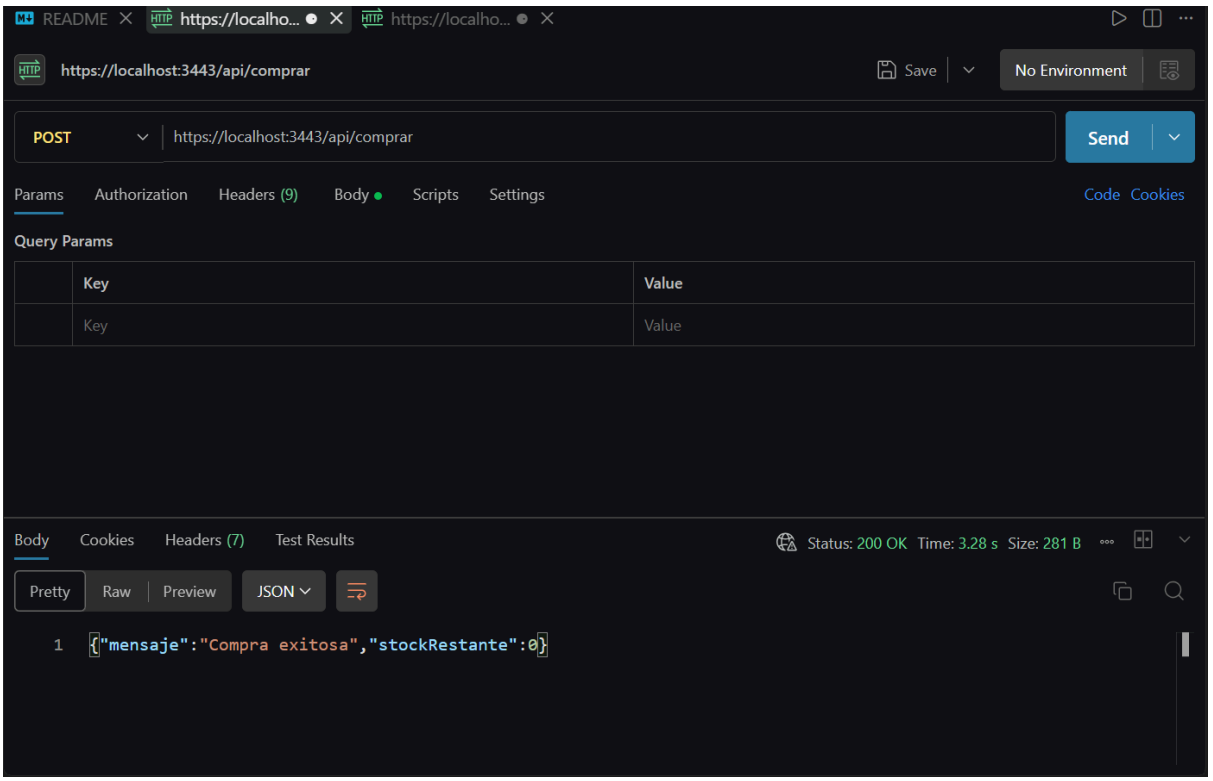


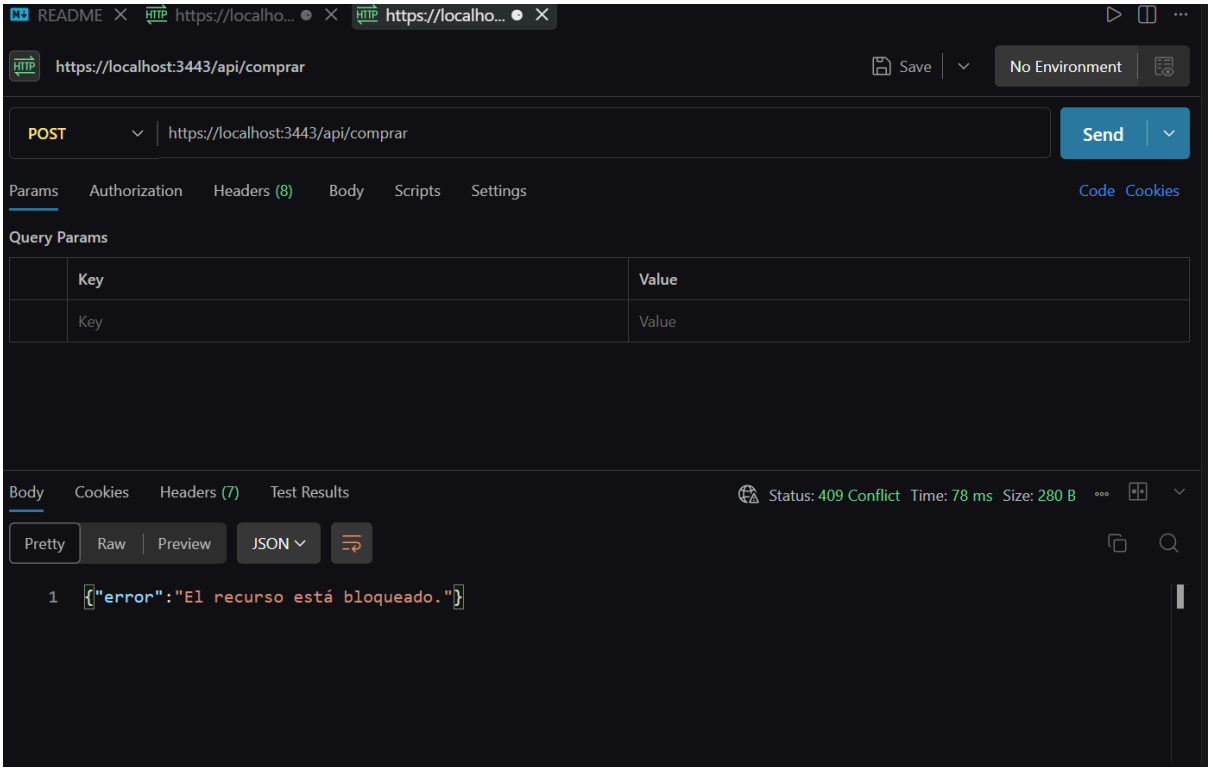


- ¿Qué diferencia existe entre OpenAPI y Swagger UI?
- OpenAPI es el estándar o la especificación escrito en formato JSON o YAML que describe teóricamente cómo funciona la API. Swagger UI es la herramienta gráfica que toma ese documento OpenAPI y genera una página web interactiva e intuitiva donde un usuario puede leerlo y ejecutar pruebas reales.
- ¿La existencia de documentación Swagger garantiza que la implementación del endpoint sea correcta? Justifique.
- No la garantiza. Swagger describe cómo *debería* comportarse la API y qué espera recibir, pero si la lógica interna del código en Express o Mongoose está defectuosa, el endpoint fallará. Swagger documenta la interfaz externa, no verifica la lógica de negocio interna.
- ¿Qué ventaja aporta disponer de un contrato de API documentado y reproducible?
- Acelera el desarrollo y la integración. Permite que equipos de Frontend o externos entiendan rápidamente qué URLs consumir, qué datos enviar y qué respuestas esperar, sin tener que leer el código fuente de Node.js ni consultar constantemente al desarrollador Backend.

Parte 4:

Punto 13:


README
https://localho...
https://localho...
https://localhost:3443/api/comprar
Save
No Environment
POST
https://localhost:3443/api/comprar
Send
Params
Authorization
Headers (9)
Body
Scripts
Settings
Code
Cookies
Query Params
Key
Value
Key
Value
Body
Cookies
Headers (7)
Test Results
Status: 200 OK
Time: 3.28 s
Size: 281 B
Pretty
Raw
Preview
JSON
1 {"mensaje":"Compra exitosa","stockRestante":0}


README
https://localho...
https://localho...
https://localhost:3443/api/comprar
Save
No Environment
POST
https://localhost:3443/api/comprar
Send
Params
Authorization
Headers (8)
Body
Scripts
Settings
Code
Cookies
Query Params
Key
Value
Key
Value
Body
Cookies
Headers (7)
Test Results
Status: 409 Conflict
Time: 78 ms
Size: 280 B
Pretty
Raw
Preview
JSON
1 {"error":"El recurso está bloqueado."}



punto 14:

- ¿Qué problema resuelve MongoDB/Mongoose que no resolvía el almacenamiento en memoria?
- La volatilidad e integridad de los datos. MongoDB asegura que los datos no se borren al apagar el servidor, y Mongoose asegura que no ingresen datos corruptos o duplicados a través de reglas estrictas y esquemas, cosas imposibles de garantizar eficientemente con arreglos nativos de JavaScript.
- ¿Qué responsabilidad cumple cada componente: MongoDB, Redis/Memurai, Winston y OpenAPI/Swagger?
    - MongoDB: Persistencia y almacenamiento a largo plazo de la base de datos de la API.
    - Redis/Memurai: Gestión rápida en memoria de la concurrencia y los candados para evitar condiciones de carrera .
    - Winston: Trazabilidad, escribiendo un historial estructurado de los eventos y errores del sistema en archivos rotativos.
    - OpenAPI/Swagger: Documentación interactiva para probar y comprender los contratos de comunicación de la API HTTP.